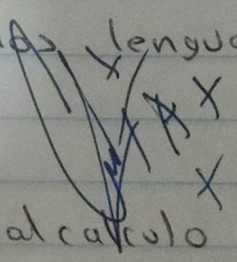
Calculo lambda Alonzo Church

El cálculo lambda es un sistema formal diseñado para investigar la definición de función, la noción de aplicación de funciones y recursión. Fue introducido por Alonzo Church y Stephen Kleene en la decada de 1930: Church usó el cálculo lambda en 1936 para resolver el Entscheidungs problem. Puede ser usado para definir de manera limpia y precisa qué es una "función computable". El interrogante de si dos expresiones del calculo lambda son equivalentes no puede ser resuelto por un algoritmo general. Esta fue la primera pregunta, incluso antes que el problema de la parada, para el cual la indecibilidad fue probada. El cálculo lambda tiene una gran influencia sobre los lenguajes funcionales, como Lisp, ML y Haskell.



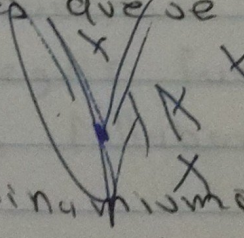
Se puede considerar al calculo lambda como el mas pequeño lenguaje universal de programación. Consiste en una regla de transformación simple (sustitución de variables) y un esquema simple para definir funciones.

El calculo lambda es universal porque cualquier función computable puede ser expresada y evaluada a través de el. Por lo tanto, es equivalente a las maquinas de Turing. Sin embargo, el cálculo lambda no hace enfasis en el uso de reglas de transformación y no considera las maquinas reales que pueden implementarlos.

Elixir

Elixir fue creado en el año 2012 por José Valim. En comparación con otros lenguajes, Elixir es una tecnología bastante nueva, sin embargo, esto no debería engañarnos, que sea un lenguaje nuevo no significa que sea frágil o que existan bugs. De hecho, Elixir se distingue por ser estable y robusto, principalmente porque los programas creados en este lenguaje se ejecutan en la maquina virtual de Erlang (BEAM)

Elixir está considerado como un lenguaje cuya finalidad es poder desarrollar todo tipo de aplicaciones escalables y mantenibles. Es decir, se dedica a no dejar obsoleto ninguno de los programas o aplicaciones que se desarrolla bajo este lenguaje.



Se considera de gran dinamismo y tambien de tipo funcional, es decir, que el usuario programador podrá asignar funciones a las distintas variables de código. Los programas que son creados y compilados en este lenguaje serán ejecutados en una máquina virtual dedicada llamada Earlang.

Erlang

Erlang está diseñado para resolver problemas que involucran concurrencia, distribución y tolerancia a fallos. A continuación se detallan algunas de las características clave que lo distinguen de otros lenguajes de programación:

Modelo de concurrencia basado en actores

Erlang utiliza un modelo de concurrencia basado en actores, donde los procesos son completamente independientes y se comunican entre sí mediante el paso de mensajes.

Tolerancia a fallos

Una de las principales razones por las que se diseñó Erlang fue para construir sistemas robustos que pudieran seguir funcionando incluso si algunas de sus partes fallaban.

Programación funcional

Erlang es un lenguaje funcional, lo que significa que fomenta el uso de funciones puras y estructuras de datos inmutables. Esto reduce los errores relacionados con el estado mutable y facilita la concurrencia, ya que los procesos no comparten memoria.

Adair Lobato Sfgo

Función de orden superior

Una función se denomina Función de orden superior si esta contiene otras funciones como parámetros de entrada o si devuelve una función como salida, es decir, las funciones que operan como otra función se conocen como funciones de orden superior en Python.

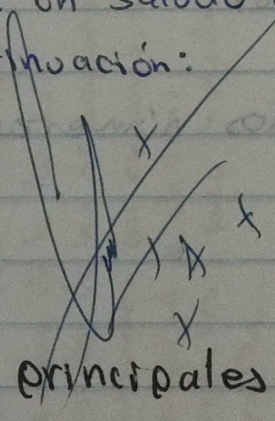
Suponga que se tiene una función, llamada high_order_function que recibe como parámetro de entrada una función:

Ejemplo 1

```
def high_order_function(fun):
    fun()
```

Y adicionalmente, en nuestro scope de trabajo de Python, tenemos otra función que recibe un saludo llamada greeting como lo vemos a continuación:

```
def greeting():
    print("Hello World")
```



Funciones de orden superior principales

map(Mapeo o transformación)

Aplica una función a cada elemento de una lista y devuelve una nueva lista con los resultados transformados.

Ejemplo:

Convertir una lista de números en sus cuadrados

[1, 2, 3] -> [1, 4, 9]

filter (Filtrado)

Evalua una condición (predicado booleano) para cada elemento de la lista y devuelve una nueva lista solo con aquellos elementos que cumplen dicha condición.

Ejemplo

Quedarse unicamente con los números pares de una lista [1, 2, 3, 4] → [2, 4]

fold / reduce (Plegado o reducción)

Recorre la lista acumulando sus elementos en un único valor final mediante una función binaria. Existen variantes como foldl (por la izquierda) y foldr (por la derecha)

Ejemplo

Sumar todos los números de una lista [1, 2, 3, 4] -> 10

	Prefija	Infija	Postfija
Definición	Son aquellas en las que el nombre de la función se escribe antes de los argumentos que recibe, permite identificar primera la operación.	Son aquellas en las que el Operador o el nombre de una función se coloca entre los argumentos, esta estructura permite representar operaciones.	Son aquellas en las que los argumentos se escriben antes del operador o de la operación que se ejecutara, esta forma de representación se relaciona con la notación polaca inversa.
Orden de los elementos	Primera se escribe el nombre de la función y despues sus argumentos, respetando el orden requerido por su definición.	Primero aparece el argumento izquierdo despues el operador o la función y finalmente el argumento derecho.	Primero aparecen los operandos y al final el operador correspondiente
Sintaxis	Se escribe colocando el nombre de la función seguido de los argumentos separados por espacios, sin necesidad de utilizar paréntesis para llamada sencilla.	Se escribe colocado el operador entre los operandos, los operadores aritmetica como +, -, * y /	No se utiliza como forma ordinaria de llamada a funciones en Haskell. Una expresión como 5 8 max no equivale a una aplicación posfija válida de max.
Limitaciones	Para algunas operaciones matematicas puede resultar menos natural que la escritura infija. Por ejemplo, div 10 2 es correcto, pero puede parecer menos familiar.	Si se desconoce la precedencia de la operadora, una expresión puede interpretarse incorrectamente además utiliza funciones ordinarias.	No es la forma habitual de escribir llamadas a funciones en Haskell. Para utilizarla directamente se requiere de escribir una estructura

Ejemplo prefijo

```
suma a b = a + b
resta a b = a - b

main = do
    print (suma 10 5)
    print (resta 10 5)
```

Ejemplo infija

```
main=do
    print (10 + 5)
    print (10 - 5)
    print (10 / 5)
    print (10 * 5)
```

Ejemplo posfija

```
evaluarSuma a b = a + b

main = do
    print (evaluarSuma 10 5)
    print (evaluarSuma 5 10)
```

D M A

Intervalo

Un conjunto de números enteros se pueden representar mediante una lista ordenada de intervalos tales que la diferencia entre el menor elemento de un intervalo y el mayor elemento de su intervalo anterior es mayor que uno.

Por ejemplo, el conjunto {2, 7, 4, 3, 9, 6} se puede representar mediante la lista de intevalos [(2,4),(6,7),(9,9)] de forma que en el primer intervalo se agrupan los números 2, 3 y 4; en el segundo, los numeros 6 y 7 el tercero, el numero 9.

```
intervalos :: [Int] -> [(Int, Int)]
```

tal que (intervalos xs) es lista ordenada de intervalos que representa al conjunto xs.

Operador

Al igual que otros lenguajes de programación, Haskell maneja de manera inteligente algunas operaciones básicas como suma, resta, multiplicación, etc. Un operador es una función binaria (de dos argumentos) cuyo nombre consiste exclusivamente en caracteres no alfanuméricos, como +, * o ==. A diferencia de las funciones estándar que usan notación de prefijo, los operadores se escriben en notación infija situándose entre sus dos operandos (ej. 2+3).

Cualquier función binaria puede convertirse en operador infijo encerrando entre comillas invertidas (ej. x `div` y), y

cualquier operador puede usarse como función prefija encerrandolo entre parentesis (ej (+) x y). La precedencia determina el orden de evaluación, siendo el operador + el de menor precedencia y (^) exponentes (^) de mayor.

Evaluación perezosa

se han visto la estrategias de evaluación perezosa e impaciente, se han comparado respecto de la terminación y el numero de pasos necesarios en las computaciones, se ha aplicado a la computación con estructuras infinitas y se han visto casos en los que se aumenta la eficiencia con evaluación estricta.

La evaluación perezosa consta de 7 puntos:

1.- Estrategias de evaluación
2.- Terminación
3.- Número de reducciones
4.- Estructuras infinitas
5.- Programacion modular
6.- Aplicación estricta
7.- Bibliografia